

Python Implementation

➤ Required Packages:

- Python 3.10
- Numpy
- Matplotlib -> box, histogram, scatter plot visualization
- scikit-learn -> SVM, LinearDiscriminantAnalysis, DecisionTreeClassifier, GaussianNB

Box Plot

'''

Box Plot Components:

- Q1 (First Quartile): 25% of the data falls below this value.
- Q3 (Third Quartile): 75% of the data falls below this value.
- Median: Middle value of the dataset. Max: Largest non-outlier value. Min: Smallest non-outlier value.
- Outlier: A point that lies outside $1.5 * IQR$ (Interquartile Range) from Q1 or Q3.

'''

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Generate random data for two classes
```

```
np.random.seed(2) # Fix the seed for reproducibility
```

```
data_class1 = np.random.normal(50, 10, 100) # Class1: mean=50, std=10, 100 samples
```

```
data_class2 = np.random.normal(60, 12, 100) # Class2: mean=60, std=12, 100 samples
```

```
# Create a box plot
```

```
plt.boxplot([data_class1, data_class2], labels=['Class1', 'Class2'], patch_artist=True)
```

```
plt.title('Box Plot')
```

```
plt.show()
```

```
# Extract boxplot statistics
```

```
for i, data in enumerate([data_class1, data_class2], start=1):
```

```
    q1 = np.percentile(data, 25) # Q1
```

```
    q3 = np.percentile(data, 75) # Q3
```

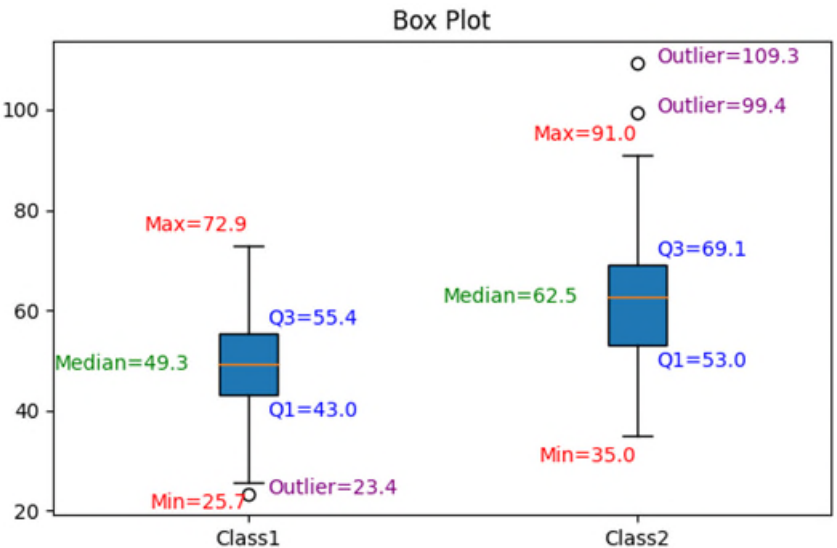
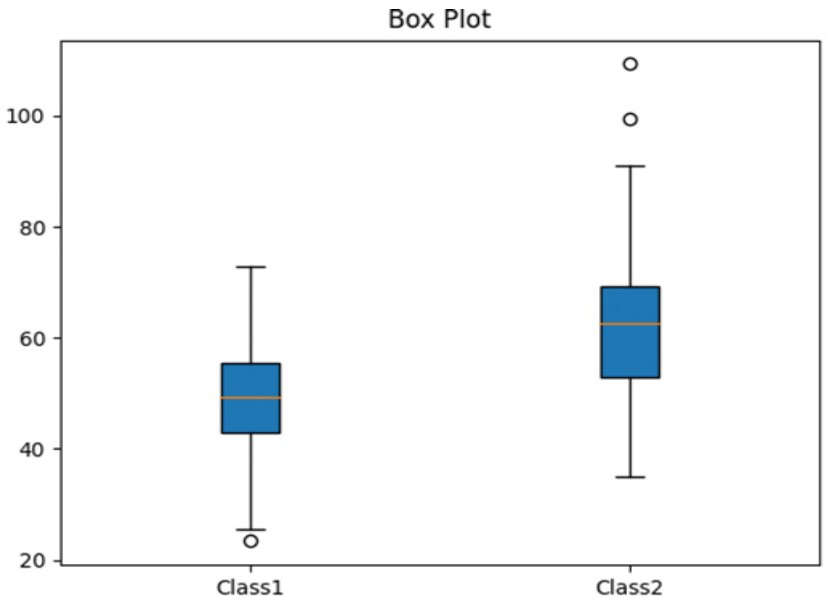
```
    median = np.median(data) # Median
```

```
    iqr = q3 - q1 # Interquartile Range
```

```
    lower_whisker = np.min(data[data >= q1 - 1.5 * iqr]) # Min (non-outlier)
```

```
    upper_whisker = np.max(data[data <= q3 + 1.5 * iqr]) # Max (non-outlier)
```

```
    outliers = data[(data < q1 - 1.5 * iqr) | (data > q3 + 1.5 * iqr)] # Outliers
```



Histogram Plot

```
"""
```

Histogram Explanation:

- A histogram shows the distribution of data.
- The x-axis represents the range of values.
- The y-axis represents the frequency (how many samples fall into each bin).
- Overlapping histograms can be used to compare two classes of data.

```
"""
```

```
import numpy as np # For numerical operations and random data generation
import matplotlib.pyplot as plt # For plotting
```

```
# Generate random data for two classes
```

```
np.random.seed(2) # Fix the seed for reproducibility
```

```
data_class1 = np.random.normal(50, 10, 100) # Class1: mean=50, std=10, 100 samples
```

```
data_class2 = np.random.normal(60, 12, 100) # Class2: mean=60, std=12, 100 samples
```

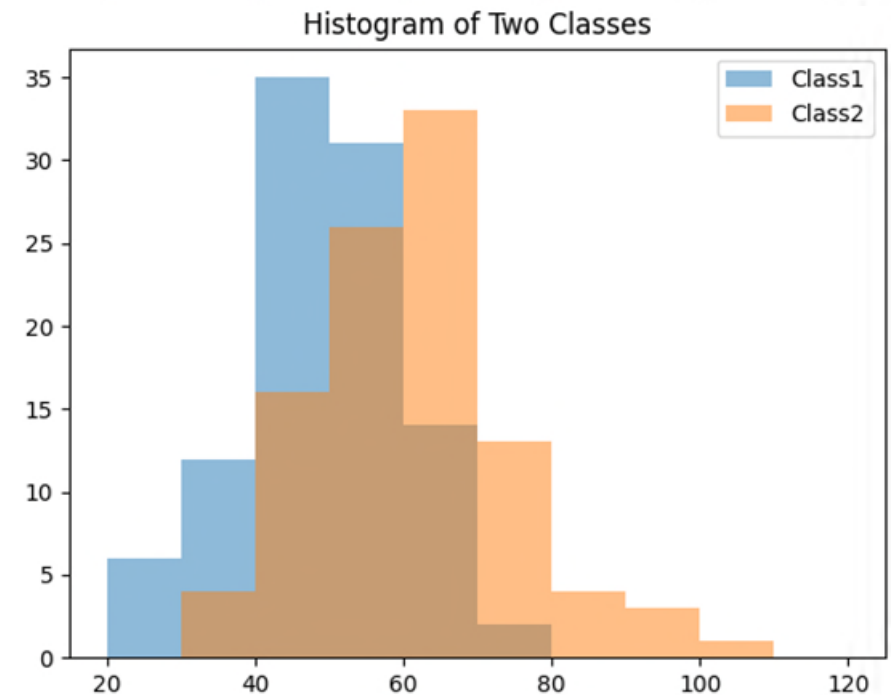
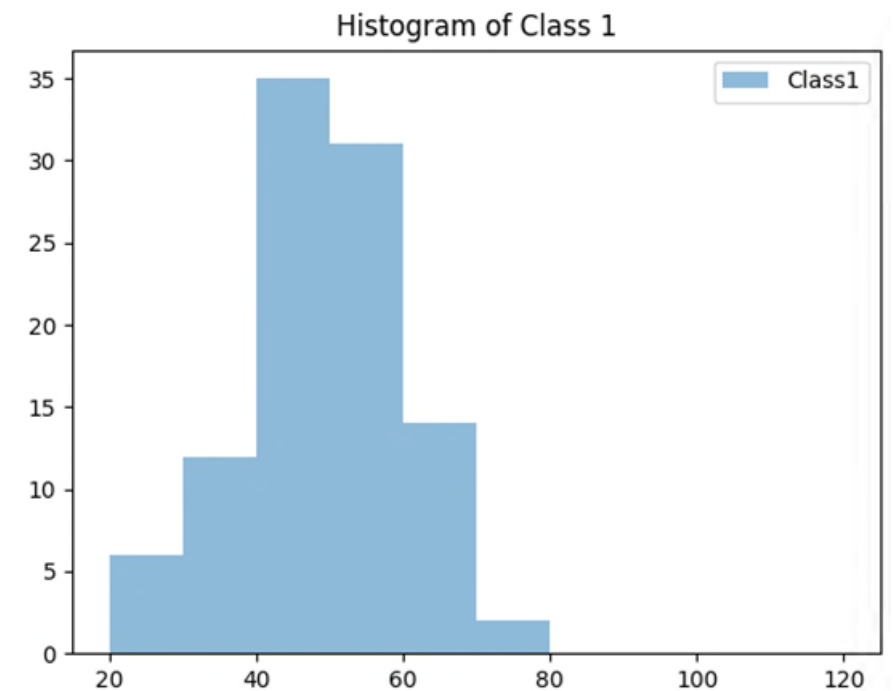
```
plt.hist(data_class1, alpha=0.5, label='Class1')
```

```
# plt.hist(data_class2, alpha=0.5, label='Class2')
```

```
plt.legend(loc='upper right')
```

```
plt.title('Histogram')
```

```
plt.show()
```



Scatter Plot

'''

Scatter Plot Explanation:

- A scatter plot shows the relationship between two variables (x and y).
- Each point represents an observation in the dataset.
- Different colors can be used to represent different classes of data.

'''

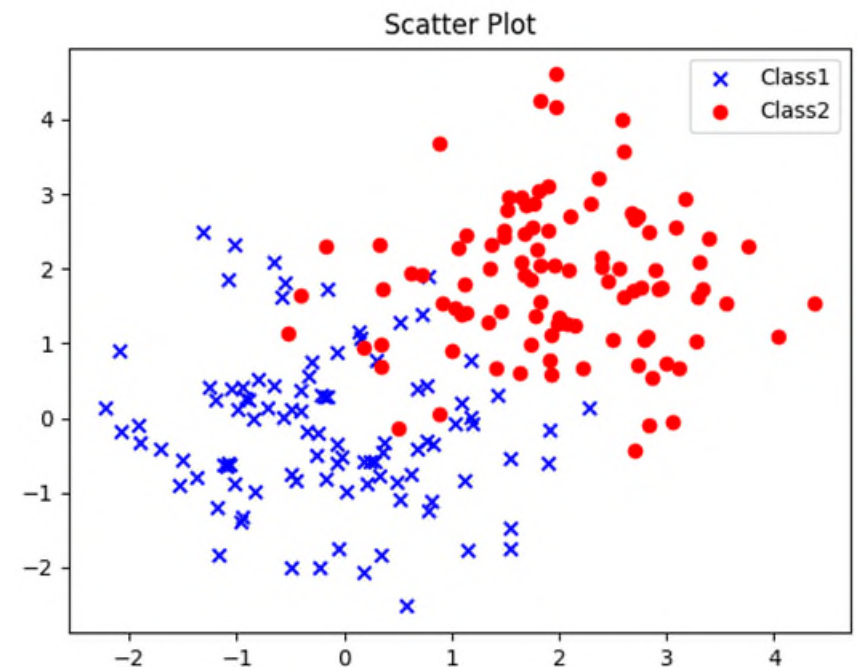
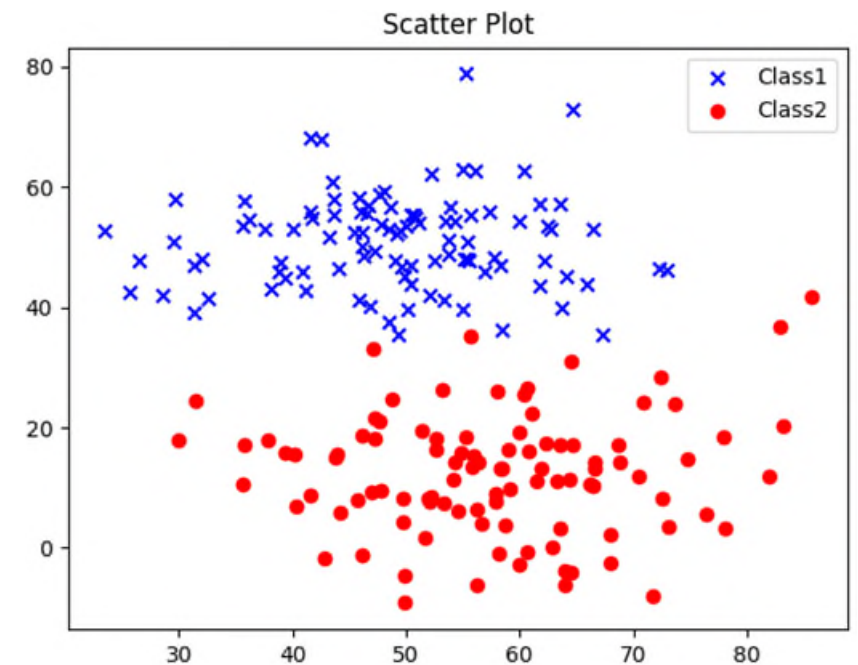
```
import numpy as np # For numerical operations and random data generation
import matplotlib.pyplot as plt # For plotting
```

```
# Generate random data for two classes
np.random.seed(2) # Fix the seed for reproducibility
```

```
x1 = np.random.normal(50, 10, 100) # X Class1: mean=50, std=10, 100 samples
y1 = np.random.normal(50, 7, 100) # Y values for Class1
```

```
x2 = np.random.normal(60, 12, 100) # X Class2: mean=60, std=12, 100 samples
y2 = np.random.normal(10, 10, 100) # Y values for Class2
```

```
plt.scatter(x1, y1, c='blue', marker='x', label='Class1')
plt.scatter(x2, y2, c='red', label='Class2')
plt.legend() # Add legend to distinguish classes
plt.title('Scatter Plot')
plt.show()
```



```
x1 = np.random.normal(0, 1, 100) y1 = np.random.normal(0, 1, 100)
x2 = np.random.normal(2, 1, 100) y2 = np.random.normal(2, 1, 100)
```

Support Vector Machine (SVM)

```
"""  
SVM Decision Boundary:  
- SVM (Support Vector Machine) is a classifier that finds the best separating hyperplane.  
"""
```

```
import numpy as np  
import matplotlib.pyplot as plt  
from sklearn import svm  
x1 = np.random.normal(50, 10, 100); y1 = np.random.normal(50, 7, 100)  
x2 = np.random.normal(60, 12, 100); y2 = np.random.normal(10, 10, 100)
```

```
# Combine data into X and labels into y  
X = np.vstack((np.column_stack((x1, y1)), np.column_stack((x2, y2))))  
y = np.array([0] * 100 + [1] * 100)
```

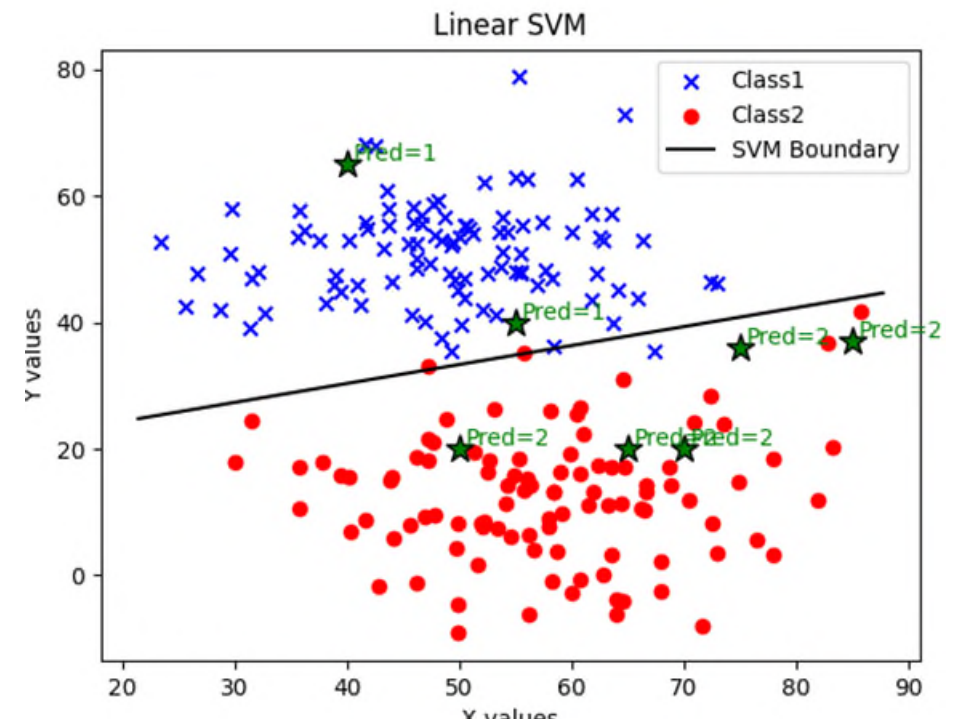
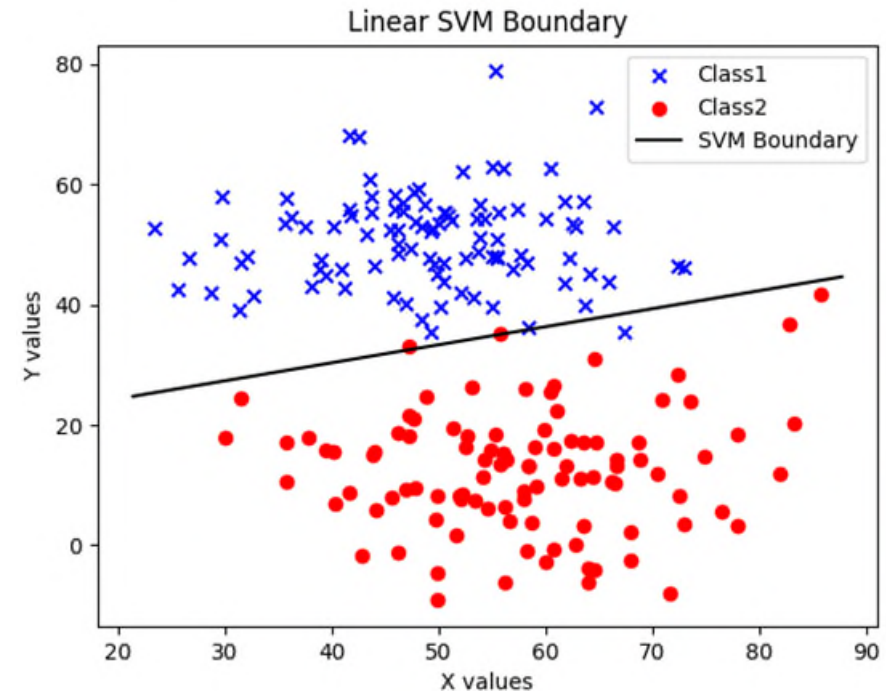
```
# Train a linear SVM classifier  
clf = svm.SVC(kernel="linear")  
clf.fit(X, y)
```

```
# Extract line parameters:  $w_1x + w_2y + b = 0$   
w = clf.coef_[0] # weight matrix (w1,w2)  
b = clf.intercept_[0] # bias b
```

```
# Make decision boundary line using min and max of x-axis  
x_min, x_max = X[:,0].min() - 2, X[:,0].max() + 2  
xx = np.linspace(x_min, x_max, 200)  
yy = -(w[0] * xx + b) / w[1] # decision boundary line
```

➤ Use the pre-trained SVM for new data evaluation

```
# 6. Define some new test points to classify  
new_points = np.array([[55, 40], [65, 20], [75, 36], [85, 37], [50, 20], [70, 20], [40, 65]])  
# Predict the class labels for the new test points  
predictions = clf.predict(new_points)
```



Linear Discriminant Function

```
"""  
Linear Discriminant Analysis (LDA):  
- Goal: Find a linear discriminant function  $g(x) = w^T x + b$   
- Principle: Maximize "between-class separation / within-class cohesion"  
"""
```

```
import numpy as np  
import matplotlib.pyplot as plt  
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
```

```
# 1. Generate random data for two classes
```

```
np.random.seed(2)  
x1 = np.random.normal(50, 10, 100) # Class1: X values  
y1 = np.random.normal(50, 7, 100)  # Class1: Y values  
x2 = np.random.normal(60, 12, 100) # Class2: X values  
y2 = np.random.normal(10, 10, 100) # Class2: Y values
```

```
# Combine into dataset
```

```
X = np.vstack((np.column_stack((x1, y1)), np.column_stack((x2, y2))))  
y = np.array([0]*100 + [1]*100) # Labels: 0 for Class1, 1 for Class2
```

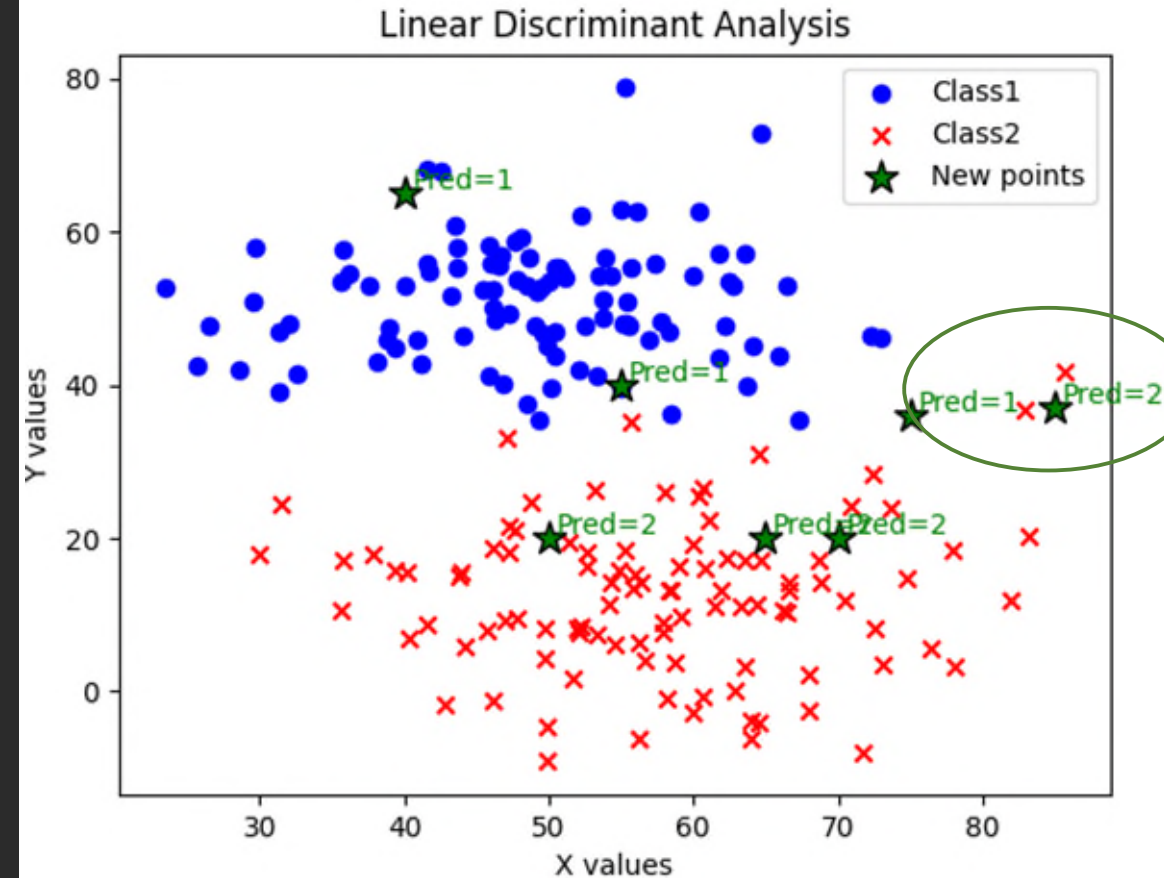
```
# 2. Train LDA model
```

```
lda = LinearDiscriminantAnalysis()  
lda.fit(X, y)
```

```
# 3. Define new test points
```

```
new_points = np.array([[55, 40], [65, 20], [75, 36], [85, 37], [50, 20], [70, 20], [40, 65]])
```

```
# Predict classes for the new points  
predictions = lda.predict(new_points)
```



Bayes Decision Rule (Naive Bayes)

```
"""
Naive Bayes Classifier:
- Decision rule: Assign a new sample x to the class that maximizes  $P(\text{class}|x)$ .
"""

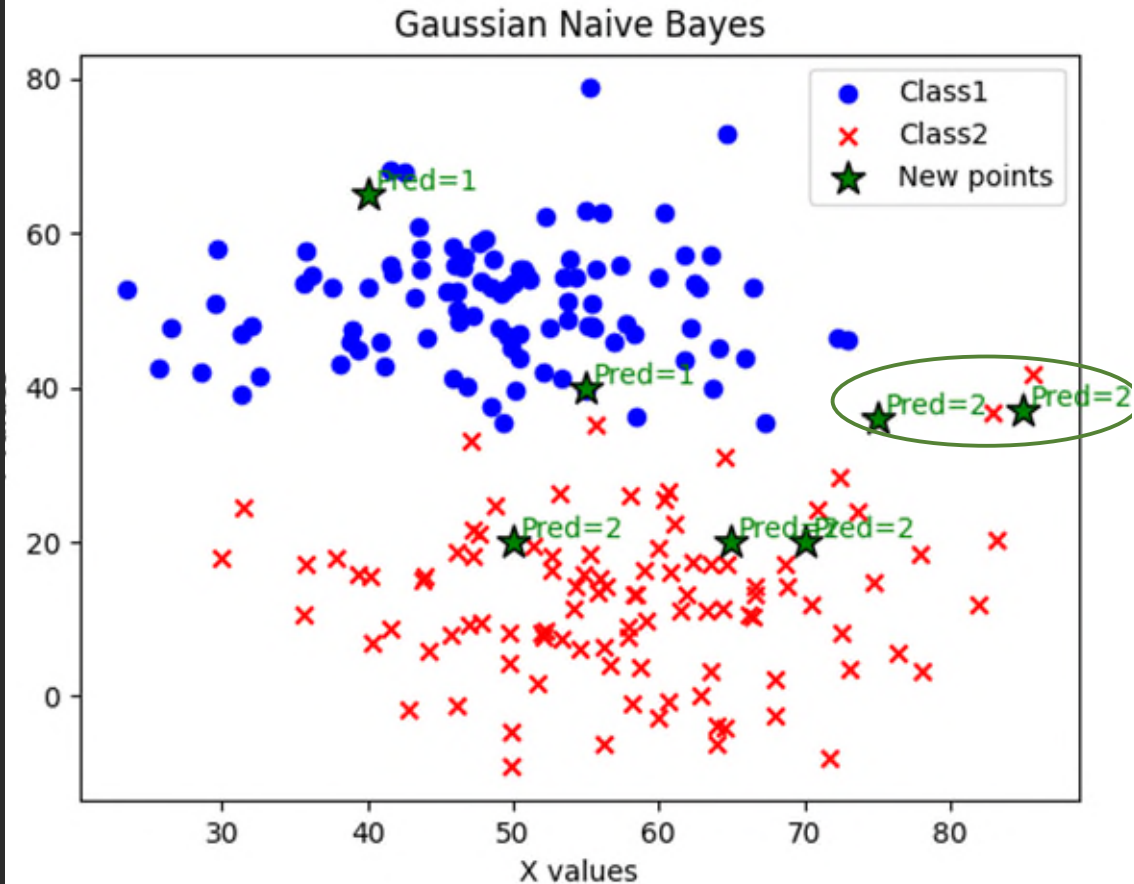
import numpy as np
import matplotlib.pyplot as plt
from sklearn.naive_bayes import GaussianNB

# 1. Generate random data for two classes
np.random.seed(2)
x1 = np.random.normal(50, 10, 100) # Class1: X values
y1 = np.random.normal(50, 7, 100) # Class1: Y values
x2 = np.random.normal(60, 12, 100) # Class2: X values
y2 = np.random.normal(10, 10, 100) # Class2: Y values

# Combine into dataset
X = np.vstack((np.column_stack((x1, y1)), np.column_stack((x2, y2))))
y = np.array([0]*100 + [1]*100) # Labels: 0 for Class1, 1 for Class2

# 2. Train Gaussian Naive Bayes classifier
gnb = GaussianNB()
gnb.fit(X, y)

# 3. Define new test points
new_points = np.array([[55, 40], [65, 20], [75, 36], [85, 37], [50, 20], [70, 20], [40, 65]])
# Predict classes for the new points
predictions = gnb.predict(new_points)
```



Decision Tree Classifier

"""

Decision Tree Classifier Explanation:

- A decision tree splits the feature space into regions by asking a sequence of yes/no questions (e.g., $x_1 < \text{threshold}$).
- Each internal node is a test on a feature, each branch is an outcome, and each leaf node assigns a class label.

"""

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
# 1. Generate random data for two classes
```

```
np.random.seed(2)
```

```
x1 = np.random.normal(50, 10, 100) # Class1: X values
```

```
y1 = np.random.normal(50, 7, 100) # Class1 Y values
```

```
x2 = np.random.normal(60, 12, 100) # Class2: X values
```

```
y2 = np.random.normal(10, 10, 100) # Class2: Y values
```

```
# Combine into dataset
```

```
X = np.vstack((np.column_stack((x1, y1)), np.column_stack((x2, y2))))
```

```
y = np.array([0]*100 + [1]*100) # Labels: 0 for Class1, 1 for Class2
```

```
# 2. Train Decision Tree classifier
```

```
tree = DecisionTreeClassifier(max_depth=3)
```

```
tree.fit(X, y)
```

```
# 3. Define new test points
```

```
new_points = np.array([[55, 40], [65, 20], [75, 36], [85, 37], [50, 20], [70, 20], [40, 65]])
```

```
# Predict classes for the new points
```

```
predictions = tree.predict(new_points)
```

